

Practical Week 5(DBMS)

MySQL Workbench

Rohita_2520090235 x

File Edit View Query Database Server Tools Scripting Help

Navigator: library_db x

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

Administration Schemas

Information

No object selected

Query 1

```
1 • CREATE DATABASE library_db;
```

2

3

Limit to 50000 rows

Output

Action Output

#	Time	Action	Message
7	20:05:53	SELECT * FROM students LIMIT 0, 50000	3 row(s) returned
8	09:38:32	CREATE DATABASE library_db	1 row(s) affected

MySQL Workbench

Rohita_2520090235

File Edit View Query Database Server Tools Scripting Help

Navigator: Query 1 library_db*

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

Administration Schemas

Information

No object selected

```

1 CREATE DATABASE library_db;
2 CREATE TABLE IF NOT EXISTS members (
3     id INT AUTO_INCREMENT PRIMARY KEY,
4     username VARCHAR(100) UNIQUE NOT NULL,
5     password_hash VARCHAR(255) NOT NULL,
6     role VARCHAR(30) NOT NULL DEFAULT 'member'
7 );
8
9 CREATE TABLE IF NOT EXISTS books (
10    id INT AUTO_INCREMENT PRIMARY KEY,
11    title VARCHAR(250) NOT NULL,
12    author VARCHAR(200) NOT NULL,
13    isbn VARCHAR(13) UNIQUE NOT NULL,
14    price DECIMAL(10,2) NOT NULL,
15    published_date DATE NOT NULL,
16    description TEXT,
17    embedding JSON
18 );
19
20
21

```

Output

Action Output

#	Time	Action	Message
8	09:38:32	CREATE DATABASE library_db	1 row(s) affected
9	09:38:53	CREATE DATABASE library_db	Error Code: 1007. Can't create database 'library_db'; database exists
10	09:40:58	CREATE TABLE IF NOT EXISTS members (id INT AUTO_INCREMENT PRIMARY KEY, username VAR...	0 row(s) affected

Object Info Session

MySQL Workbench

Rohita_2520090235

File Edit View Query Database Server Tools Scripting Help

Navigator: Query 1 library_db*

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

Administration Schemas

Information

No object selected

```

1 CREATE DATABASE library_db;
2 CREATE TABLE IF NOT EXISTS members (
3     id INT AUTO_INCREMENT PRIMARY KEY,
4     username VARCHAR(100) UNIQUE NOT NULL,
5     password_hash VARCHAR(255) NOT NULL,
6     role VARCHAR(30) NOT NULL DEFAULT 'member'
7 );
8
9 CREATE TABLE IF NOT EXISTS books (
10    id INT AUTO_INCREMENT PRIMARY KEY,
11    title VARCHAR(250) NOT NULL,
12    author VARCHAR(200) NOT NULL,
13    isbn VARCHAR(13) UNIQUE NOT NULL,
14    price DECIMAL(10,2) NOT NULL,
15    published_date DATE NOT NULL,
16    description TEXT,
17    embedding JSON
18 );
19
20
21

```

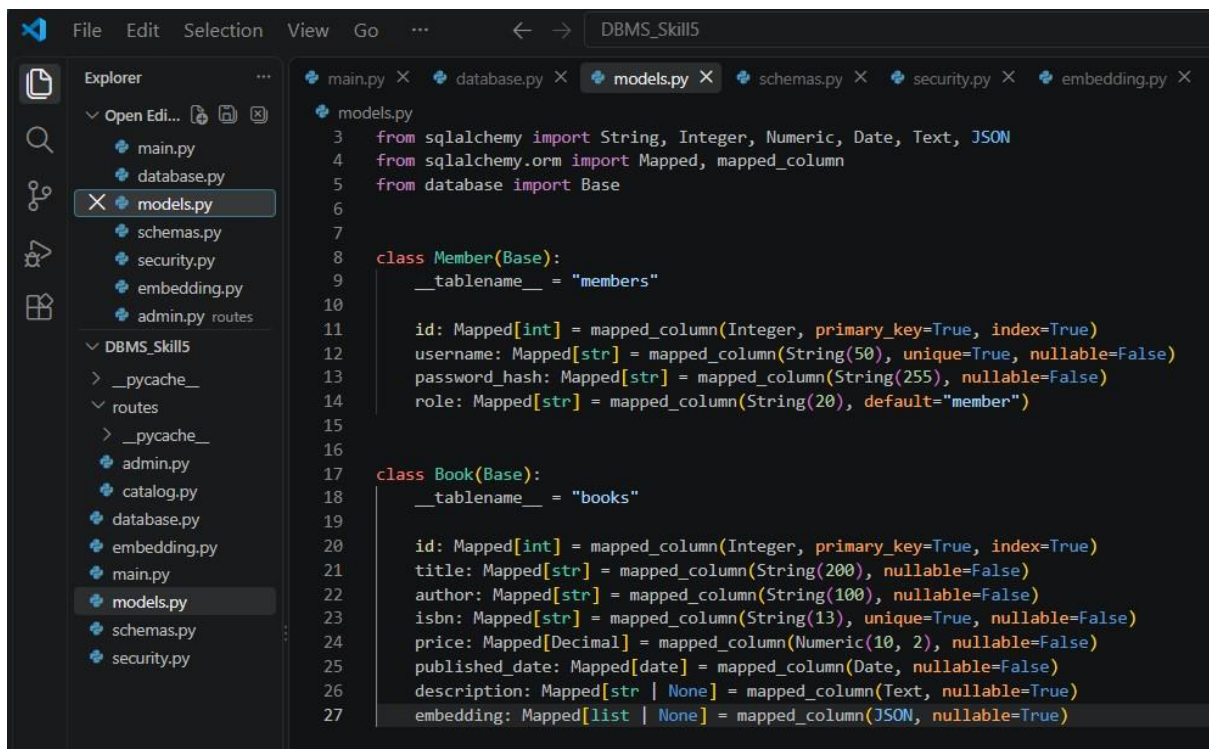
Output

Action Output

#	Time	Action	Message
9	09:38:53	CREATE DATABASE library_db	Error Code: 1007. Can't create database 'library_db'; database exists
10	09:40:58	CREATE TABLE IF NOT EXISTS members (id INT AUTO_INCREMENT PRIMARY KEY, username VAR...	0 row(s) affected
11	09:41:27	CREATE TABLE IF NOT EXISTS books (id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(2...	0 row(s) affected

Object Info Session

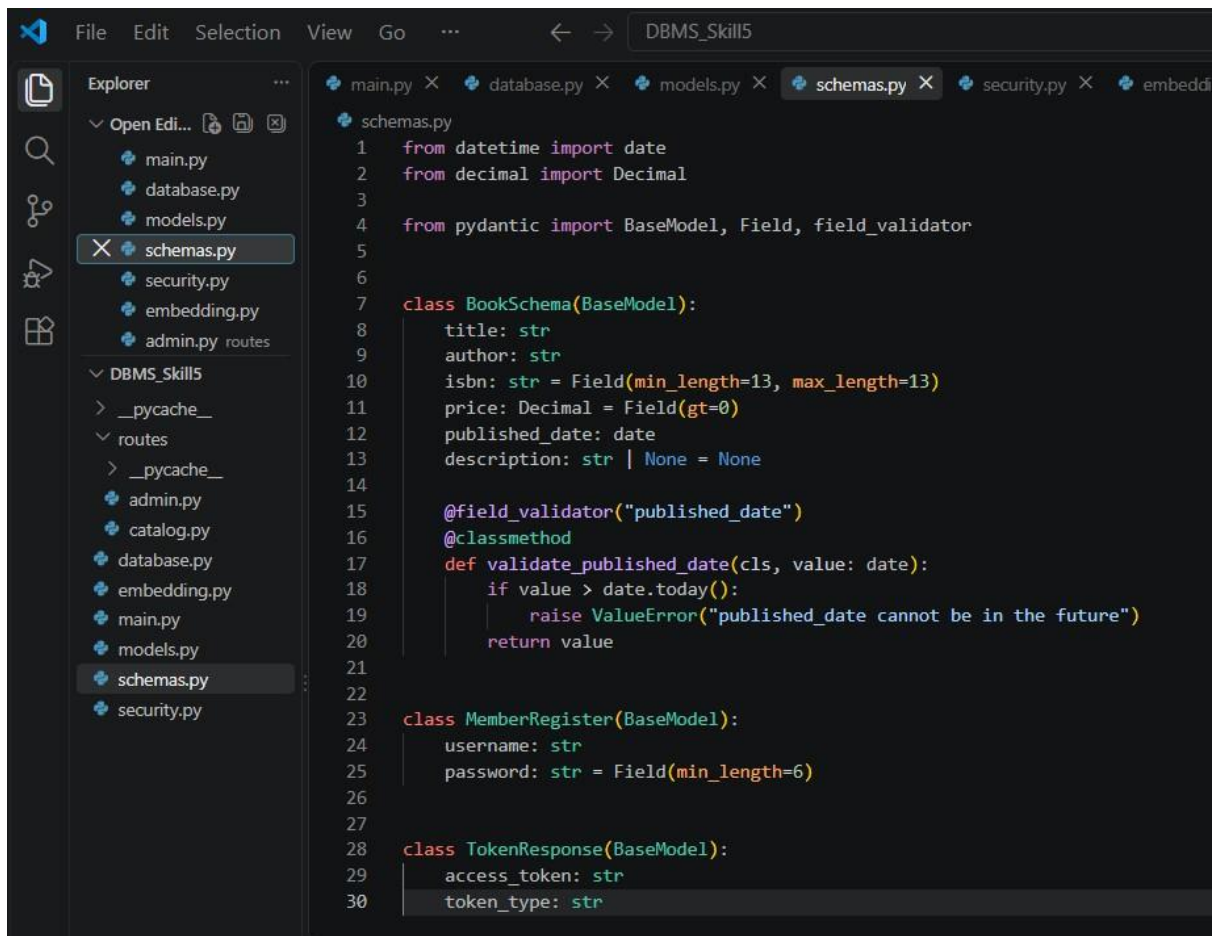
```
PS C:\library_api> python -m pip install fastapi uvicorn
>> python -m pip install "python-jose[cryptography]"
>> python -m pip install passlib[bcrypt]
● >> python -m pip install python-multipart
```



```
File Edit Selection View Go ... DBMS_Skill5

main.py X database.py X models.py X schemas.py X security.py X embedding.py X

models.py
3 from sqlalchemy import String, Integer, Numeric, Date, Text, JSON
4 from sqlalchemy.orm import Mapped, mapped_column
5 from database import Base
6
7
8 class Member(Base):
9     __tablename__ = "members"
10
11     id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
12     username: Mapped[str] = mapped_column(String(50), unique=True, nullable=False)
13     password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
14     role: Mapped[str] = mapped_column(String(20), default="member")
15
16
17 class Book(Base):
18     __tablename__ = "books"
19
20     id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
21     title: Mapped[str] = mapped_column(String(200), nullable=False)
22     author: Mapped[str] = mapped_column(String(100), nullable=False)
23     isbn: Mapped[str] = mapped_column(String(13), unique=True, nullable=False)
24     price: Mapped[Decimal] = mapped_column(Numeric(10, 2), nullable=False)
25     published_date: Mapped[date] = mapped_column(Date, nullable=False)
26     description: Mapped[str | None] = mapped_column(Text, nullable=True)
27     embedding: Mapped[list | None] = mapped_column(JSON, nullable=True)
```



```
File Edit Selection View Go ... DBMS_Skill5

main.py X database.py X models.py X schemas.py X security.py X embedd...

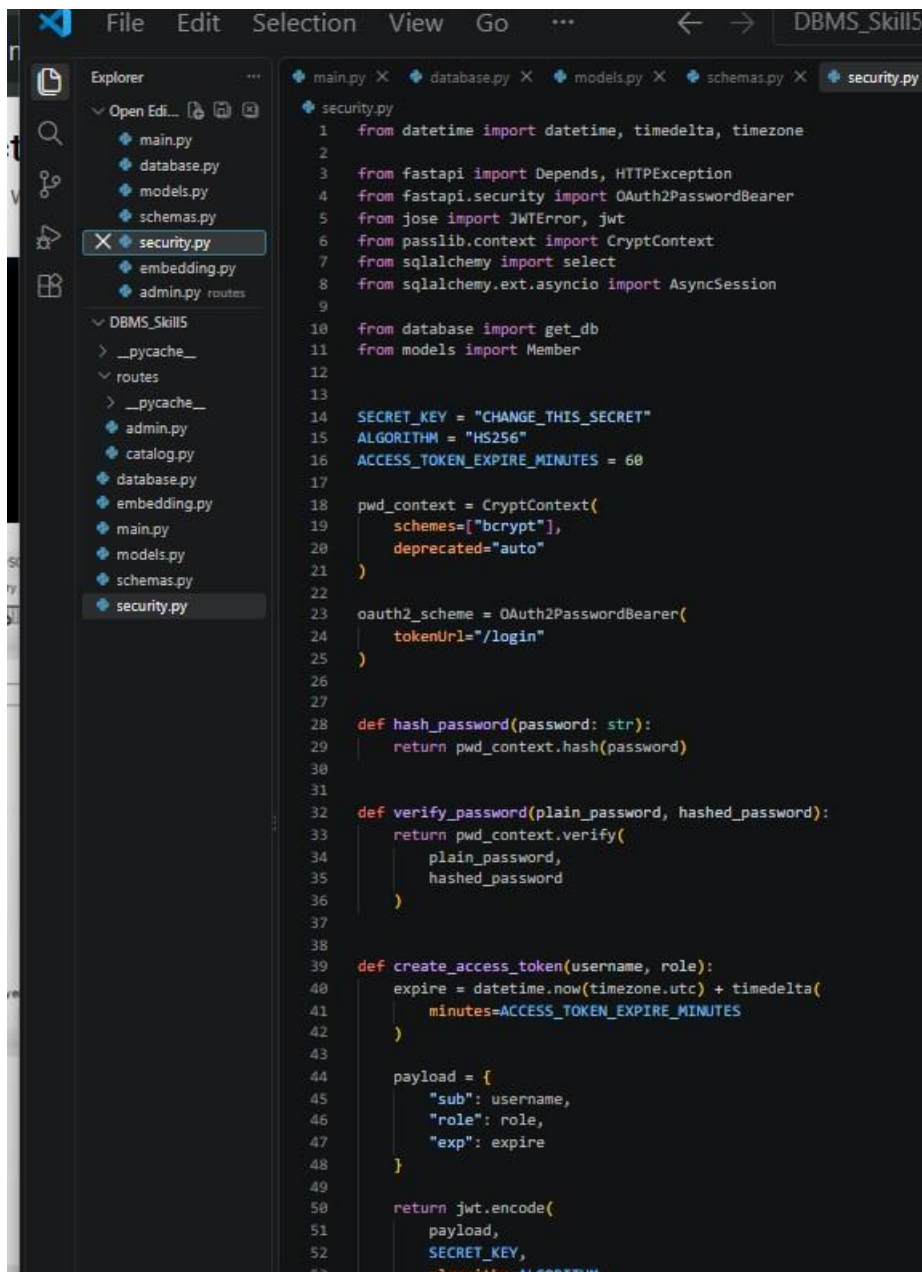
schemas.py
1 from datetime import date
2 from decimal import Decimal
3
4 from pydantic import BaseModel, Field, field_validator
5
6
7 class BookSchema(BaseModel):
8     title: str
9     author: str
10     isbn: str = Field(min_length=13, max_length=13)
11     price: Decimal = Field(gt=0)
12     published_date: date
13     description: str | None = None
14
15     @field_validator("published_date")
16     @classmethod
17     def validate_published_date(cls, value: date):
18         if value > date.today():
19             raise ValueError("published_date cannot be in the future")
20         return value
21
22
23 class MemberRegister(BaseModel):
24     username: str
25     password: str = Field(min_length=6)
26
27
28 class TokenResponse(BaseModel):
29     access_token: str
30     token_type: str
```

This screenshot shows the Visual Studio Code editor with the `embedding.py` file open. The Explorer sidebar on the left shows the project structure, with `embedding.py` selected under the `DBMS_Skills5` folder. The main editor displays the following Python code:

```
3
4 VECTOR_DIM = 384
5
6
7 def text_to_vector(text: str) -> list[float]:
8     values = []
9
10    for i in range(VECTOR_DIM):
11        digest = hashlib.sha256(
12            f"{text.lower()}{i}".encode()
13        ).digest()
14
15        number = int.from_bytes(
16            digest[:4],
17            byteorder="big"
18        )
19
20        values.append(
21            (number / 2**32) * 2 - 1
22        )
23
24    magnitude = math.sqrt(
25        sum(x * x for x in values)
26    )
27
28    return [x / magnitude for x in values]
```

This screenshot shows the Visual Studio Code editor with the `main.py` file open. The Explorer sidebar on the left shows the project structure, with `main.py` selected under the `DBMS_Skills5` folder. The main editor displays the following Python code:

```
1 from fastapi import (
2     BackgroundTasks,
3     Depends,
4     FastAPI,
5     HTTPException,
6     Query,
7     status
8 )
9
10 from fastapi.security import OAuth2PasswordRequestForm
11
12 from sqlalchemy import select
13 from sqlalchemy.ext.asyncio import AsyncSession
14
15 from database import Base, engine, get_db
16 from embedding import text_to_vector
17 from models import Book, Member
18 from schemas import BookSchema, MemberRegister, TokenResponse
19
20 from security import (
21     create_access_token,
22     get_current_member,
23     hash_password,
24     verify_password
25 )
26
27 import math
28
29
30 app = FastAPI()
```



```
1 from datetime import datetime, timedelta, timezone
2
3 from fastapi import Depends, HTTPException
4 from fastapi.security import OAuth2PasswordBearer
5 from jose import JWTError, jwt
6 from passlib.context import CryptContext
7 from sqlalchemy import select
8 from sqlalchemy.ext.asyncio import AsyncSession
9
10 from database import get_db
11 from models import Member
12
13
14 SECRET_KEY = "CHANGE_THIS_SECRET"
15 ALGORITHM = "HS256"
16 ACCESS_TOKEN_EXPIRE_MINUTES = 60
17
18 pwd_context = CryptContext(
19     schemes=["bcrypt"],
20     deprecated="auto"
21 )
22
23 oauth2_scheme = OAuth2PasswordBearer(
24     tokenUrl="/login"
25 )
26
27
28 def hash_password(password: str):
29     return pwd_context.hash(password)
30
31
32 def verify_password(plain_password, hashed_password):
33     return pwd_context.verify(
34         plain_password,
35         hashed_password
36     )
37
38
39 def create_access_token(username, role):
40     expire = datetime.now(timezone.utc) + timedelta(
41         minutes=ACCESS_TOKEN_EXPIRE_MINUTES
42     )
43
44     payload = {
45         "sub": username,
46         "role": role,
47         "exp": expire
48     }
49
50     return jwt.encode(
51         payload,
52         SECRET_KEY,
53         algorithm=ALGORITHM
```

